■■ NavTools: Assistive Gaze Tracking & Voice Assistant

A Hands-Free Assistive Control Suite using MediaPipe Gaze Tracking and Offline Voice Command Orchestration

Group No. 7 · 8th Semester Major Project · 2026

■ Table of Contents

■ Overview

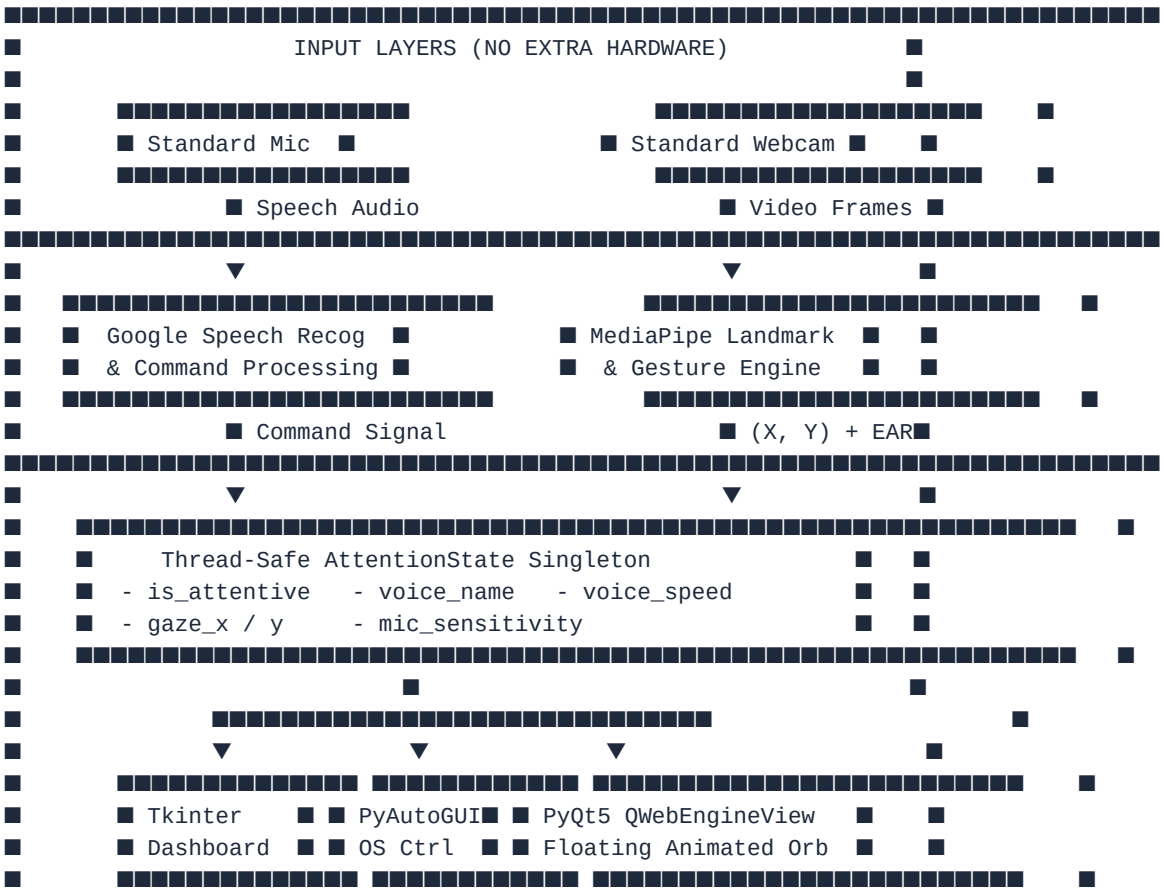
NavTools is a completely non-invasive, premium assistive control suite that empowers individuals with physical impairments to operate computers hands-free. The system merges **webcam-based eye gaze tracking** with an **offline voice-activated assistant (Jim)**.

Unlike legacy biosignal-based interfaces (like EEG/EOG), NavTools requires **zero extra hardware**. It leverages standard computer webcams and microphones to deliver smooth cursor movement, rich facial-gesture-based clicking, and highly responsive voice command execution.

Key Highlights

Feature	Details
■■ Gaze Control	MediaPipe Face Mesh landmark tracking for cursor positioning
■ Gestures	Blink and Wink gesture recognition for clicks (Left, Right, Double)
■■ Voice Command	120+ pre-configured offline/online speech macros (Jim Assistant)
■■ Configuration	Unified dark-mode Tkinter settings panel (Voice, Gaze, General tabs)
■ Attention Gating	Thread-safe inter-module synchronization via shared state
■■ TTS System	High-fidelity Kokoro ONNX voice generation with local SAPI5 fallback
■ Floating Orb	PyQt5/QWebEngineView transparent animated orb with drag support
■ YouTube Music	Dedicated voice controls for YouTube Music tab (play, next, mute, like, shuffle)

■■ System Architecture



■■■

■ Core Modules

1. Camera-Based Gaze Tracking

The **Gaze Tracker** acts as a virtual mouse by tracking the user's face and eyes using MediaPipe's High-Fidelity Landmark Model (`face_landmarker.task`).

- **Exponential Moving Average (EMA) Smoothing:** Eliminates natural micro-saccades and camera jitter to provide smooth, organic cursor movement.
- **Eye Aspect Ratio (EAR) Clicking:** Detects natural blinks and winks for full mouse emulation:
  - ■ ■ **Left Eye Wink:** Executes a **Left Mouse Click**.
  - ■ ■ **Right Eye Wink:** Executes a **Right Mouse Click**.
  - ■ ■■ **Double Blink:** Executes a **Double-Click**.
  - ■ ■■ **Extended Hold Blink:** Triggers mouse **Click & Drag**.

2. Jim Interactive Voice Assistant

Jim is an offline-first interactive digital companion that processes vocal commands. It features:

- **120+ Voice Commands:** Spanning app launches, browser/tab navigation, file operations, system controls, YouTube Music, window management, and more.
- **Smart Calculation Parser:** Extracts dynamic spoken expressions (e.g., *"calculate fifteen times three plus four"*) and executes them safely.
- **Smart Web & Local Search:** Supports targeted phrases (e.g., *"search Google for python guides"*, *"find file design_specifications"*, *"type hello world"*).
- **Positional Link/File Clicking:** Commands like *"click first result"*, *"click second link"*, *"open third file"* work in browsers and File Explorer.
- **Kokoro TTS Engine:** Integrates lightweight ONNX models to produce crystal-clear human-like speech offline.
- **Dynamic Mic Sensitivity:** Reads `attention.mic_sensitivity` before each listen call for real-time adjustment.

3. Animated Orb UI (PyQt5 + QWebEngineView)

The system renders a **floating, frameless, fully transparent orb** on your screen using PyQt5's `QMainWindow` + `QWebEngineView`:

- **Frameless & Transparent:** `Qt.FramelessWindowHint | Qt.WA_TranslucentBackground` — no border, pure floating orb.
- **Always On Top:** `Qt.WindowStaysOnTopHint` — remains visible above all other windows.
- **Drag-to-Move:** Click and drag the orb to any screen position; delta coordinates are relayed via `console.log("ORB_DRAG:dx,dy")` to the Python bridge.

- **Color-coded States:**
  - ■ ■ **Idle** — Dim pulsing blue (waiting for wake word)
  - ■ ■ **Listening** — Bright cyan/green pulse (hearing your command)
  - ■ ■ **Speaking** — Amber glow (Jim is responding)
- **Double-click** → Opens the Tkinter Settings Dashboard.
- **Right-click** → Exits the entire application gracefully.

4. General Settings Tab

A new **General Settings** tab in the Control Center Dashboard provides runtime-adjustable parameters:

Setting	Description	Default
Voice Profile	Selects TTS voice: "Soft Female (Zira)" or "Natural Female (Sarah)"	Soft Female (Zira)
Speaking Speed	Adjusts voice speed from 0.5× to 2.0×	1.0×
Microphone Sensitivity	Energy threshold for voice pick-up (50–1000); lower = more sensitive	300

Changes apply instantly — the voice assistant reads these from `AttentionState` before each speak/listen cycle.

■ Software Setup

Prerequisites

- **OS:** Windows 10/11 (preferred for pywin32 API system hooks and SAPI5 TTS).
- **Python:** 3.9 through 3.11.
- **Webcam & Microphone:** Any standard built-in or external USB device.

Installation

Option A — Automated Installer (Recommended)

Simply double-click `install.bat`. It will:

- Check for Python in PATH.
- Create a `.venv` virtual environment.
- Upgrade pip and install all dependencies from `requirements.txt`.
- Download the MediaPipe Face Landmarker model (~80MB) into `data/`.

Option B — Manual Setup

- **Clone the Repository:**

```bash
git clone https://github.com/JatinderpalSingh9321/Assistant-Project-for-Paralysed-people.git
cd Assistant-Project-for-Paralysed-people
```

- **Set Up Virtual Environment:**

```bash
python -m venv .venv
.\.venv\Scripts\activate
```

- **Install Dependencies:**

```bash
pip install -r requirements.txt
```

- **Download Face Landmarker Model:**

```bash
# PowerShell
Invoke-WebRequest -Uri "https://storage.googleapis.com/mediapipe-models/face_landmarker/face_landmarker/float16/1/face_landmarker.task" -OutFile "data\face_landmarker.task"
```

- *(Optional)* **Download Kokoro TTS Models:**

```bash
python scripts/download_kokoro.py
```

■ How to Run

Method 1: Quick Start (run.bat)

Double-click `run.bat` — it automatically uses the `.venv` Python and launches the full Control Center dashboard:

```
run.bat
```

Method 2: Standalone Multimodal Console Launcher

Runs both Eye Gaze Tracker and Voice Assistant as independent threads with console logs:

```
# Basic start (Gaze + Voice)
python -m src.multimodal_launcher

# Start with a real-time webcam preview panel for camera calibration
python -m src.multimodal_launcher --preview

# Disable gaze module (voice only)
python -m src.multimodal_launcher --no-gaze
```

Method 3: Control Center Dashboard

Launches the full dark-themed GUI control panel with three settings tabs:

```
python -m src.gui_app
```

The dashboard includes:

- ■■ **Voice Assistant tab** — Start/Stop Jim, enable Attention Gating.
- ■■ **Gaze Tracker tab** — Configure camera index, EMA smoothing, toggle live preview.
- ■■ **General Settings tab** — Select voice profile, speaking speed, microphone sensitivity.

■ Unified State Management

To prevent cursor drift or voice triggers when the user is not actively looking at the screen, NavTools implements a thread-safe singleton state class, `AttentionState`.

`AttentionState` Properties

Property	Type	Description
`is_attentive`	`bool`	Whether the user is focused on the screen (staleness check after 2s)
`gaze_x / gaze_y`	`float`	Normalized gaze coordinates (0.0–1.0)
`voice_name`	`str`	Active TTS voice profile (`"zira"` or `"sarah"`)
`voice_speed`	`float`	Speaking speed multiplier (0.5–2.0)
`mic_sensitivity`	`int`	Recognizer energy threshold (50–1000)

If **Attention Gating** is enabled:

- **Attentive Calibration:** The eye gaze tracker checks if the user's pupils are focused on the monitor.

- **Signal Gating:** If the user looks away, `attention.is_attentive` is set to `False`.
- **Trigger Lockout:** The Voice Assistant pauses microphone streaming and cursor positioning freezes.
- **Resumption:** Simply looking back at the screen immediately resumes all controls.

■■ Voice Assistant Reference Guide

Jim listens for the wake phrase **"wake up Jim"** or **"hey Jim"**. Once activated, speak any of the following command classes:

Browser & Tab Navigation

Command	Action
"open google" / *"open youtube"*	Launches browser to specified URLs
"new tab" / *"close tab"*	Manages browser tabs
"next tab" / *"previous tab"*	Cycles through active tabs
"go back" / *"go forward"* / *"refresh page"*	Standard page navigation
"click first result" / *"click second link"*	Clicks nth link on current page

Local Application Launchers

- *"open calculator"* / *"open notepad"* / *"open task manager"* / *"open terminal"*
- *"open windows settings"* / *"open device manager"* / *"open remote desktop"*
- *"open word"* / *"open excel"* / *"open powerpoint"* / *"open outlook"*
- *"open steam"* / *"open discord"* / *"open spotify"* / *"open vs code"* / *"open photoshop"*
- *"open downloads"* / *"open documents"* / *"open pictures"* / *"open desktop"*

Windows & System Controls

- *"minimize window"* / *"maximize window"* / *"snap left"* / *"snap right"* / *"full screen"*
- *"scroll down"* / *"scroll up"* / *"page up"* / *"page down"* / *"go to top"* / *"go to bottom"*
- *"take screenshot"* — Windows Snip & Sketch overlay.
- *"lock screen"* — Instantly locks the Windows session.
- *"volume up"* / *"volume down"* / *"mute"* — System-wide audio control.
- *"copy"* / *"paste"* / *"undo"* / *"redo"* / *"select all"*

Global Media Controls

- *"play"* / *"pause"* / *"play pause"* — Windows global media keys.
- *"next song"* / *"previous song"* / *"skip track"* — Track navigation.

YouTube Music Controls

Command	Action
"play music" / "pause music"	Toggle playback on YouTube Music tab
"next music" / "previous music"	Skip tracks
"mute music" / "unmute music"	Mute/unmute
"like this song" / "dislike this song"	Rate current track
"shuffle music" / "shuffle on"	Toggle shuffle
"volume up music" / "volume down music"	Adjust music volume
"open youtube music"	Opens music.youtube.com in browser

File Explorer Controls

- *"open first file" / "open second file" ... "open fifth file" / "open last file"*
- *"select first file" / "next file" / "previous file"*
- *"rename file" / "delete file" / "copy file" / "paste file" / "new folder"*

Smart Dynamic Handlers

- *"calculate [expression]"* — Solves equations vocally.
- *"search for [query]"* — Smart search (Google or YouTube Music depending on context).
- *"google search for [query]"* — Always searches Google.
- *"search in file explorer for [name]"* — Opens File Explorer with query.
- *"find file [filename]"* — File Explorer structured search.
- *"type [text]"* — Types text directly into the active cursor field.
- *"open [app name]" / "close [app name]"* — Dynamic app launch/close.
- *"play [song/artist name]"* — Searches YouTube Music.

Gaze Tracker Controls (via Voice)

- *"launch eye tracking" / "start eye cursor"* — Starts the gaze tracking module.
- *"stop eye tracking" / "close eye cursor"* — Stops the gaze tracking module.

Settings & Assistant Controls

- *"open settings" / "show dashboard" / "open control center"* — Opens the Control Center.
- *"close settings" / "hide panel"* — Hides the Control Center.
- *"help" / "what can you do"* — Jim lists available commands.
- *"stop listening" / "go to sleep"* — Puts Jim to sleep (wake phrase restores).
- *"close the assistant" / "exit application"* — Graceful full shutdown.

■ Troubleshooting

Issue	Likely Cause	Solution
Jittery cursor	Low room lighting or low camera FPS	Increase ambient lighting; increase gaze `SMOOTHING` value (e.g., `0.9`) in the Gaze Tracker settings tab.
Blinks not clicking	Face landmark model file is missing	Ensure `data/face_landmarker.task` is present. Run `install.bat` or download manually.
Voice command lag	High ambient microphone noise	Move to a quieter area or raise `Microphone Sensitivity` in General Settings.
Jim not waking up	Energy threshold too high	Lower `Microphone Sensitivity` in the General Settings tab (try 150–250).
Kokoro TTS unavailable	Model weights missing or not installed	System auto-falls back to Windows SAPI5. Run `python scripts/download_kokoro.py` to install Kokoro.
Orb not appearing	PyQt5/QWebEngine not installed	Run `pip install PyQt5 PyQtWebEngine` or re-run `install.bat`.
`run.bat` **fails**	`.venv` not created	Run `install.bat` first, or create `.venv` manually and install `requirements.txt`.

■ Project Structure

```
Assistant-Project-for-Paralysed-people/
■■■ src/
■   ■■■ gui_app.py              # Main app: PyQt5 Orb + Tkinter Dashboard
■   ■■■ voice_assistant.py      # Jim: 120+ voice commands, Kokoro/SAPI5 TTS
■   ■■■ attention_state.py      # Thread-safe shared state singleton
■   ■■■ gaze_tracker.py         # MediaPipe face landmark gaze + EAR clicking
■   ■■■ multimodal_launcher.py  # CLI launcher (gaze + voice threads)
■   ■■■ eye_tracker.py          # Low-level eye tracking utilities
■   ■■■ gaze_mouse.py           # PyAutoGUI mouse control bridge
■   ■■■ utils.py                # Logger and path utilities
■■■ scripts/
■   ■■■ generate_report.py      # B.Tech major project report generator (DOCX)
■   ■■■ download_kokoro.py      # Kokoro ONNX model downloader
■   ■■■ test_kokoro.py          # Kokoro TTS test script
■■■ data/
■   ■■■ face_landmarker.task    # MediaPipe model (downloaded by install.bat)
■■■ models/
■   ■■■ kokoro/                 # Kokoro ONNX model weights (optional)
■■■ orb.html                   # Animated orb HTML/CSS/JS
■■■ requirements.txt           # Python dependencies
■■■ install.bat                # One-click installer
```

```
■■■ run.bat                    # One-click launcher (uses .venv)
■■■ BCI_NavTools_Major_Project_Report.docx  # Full B.Tech project report
```

■ License

This project is licensed under the MIT License.

Assistive Technology made elegant, lightweight, and accessible to everyone. ■■■■

Group 7 — 8th Semester Major Project, 2026